

Министерство образования Республики Беларусь Учреждение
образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине «ОМО»
Тема: «Введение в
нейронные сети: построение
многослойного перцептрона»

Выполнила:
Студентка 3 курса
Группы АС-66
Петрусевич А.А.
Проверил:
Крощенко А.А.

Брест 2025

Цель работы: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации.

Ход работы Вариант 11

Общий план для всех вариантов:

1. Импорт библиотек и подготовка данных

- импортируйте torch, torch.nn, torch.optim, а также sklearn для загрузки данных и их предобработки;
- загрузите датасет, выполните стандартизацию (StandardScaler) и кодирование признаков;
- разделите данные на обучающую и тестовую выборки;
- преобразуйте данные (признаки и метки) в тензоры PyTorch: `torch.tensor(X_train, dtype=torch.float32)`.

2. Определение архитектуры нейронной сети

- создайте класс, наследуемый от `torch.nn.Module`;
- в методе `__init__` определите все слои, которые будете использовать (например, `nn.Linear`, `nn.ReLU`, `nn.Dropout`);
- в методе `forward` опишите последовательность применения слоев к входным данным.

3. Инициализация модели, функции потерь и оптимизатора

- создайте экземпляр вашей модели: `model = MLP()`;
- определите функцию потерь. Для бинарной классификации используйте `nn.BCEWithLogitsLoss`, для многоклассовой – `nn.CrossEntropyLoss`;
- определите оптимизатор:
`optimizer = torch.optim.Adam(model.parameters(), lr=0.001)`.

4. Написание цикла обучения (Training Loop)

- запустите цикл на определенное количество эпох;
- внутри цикла:
 1. переведите модель в режим обучения: `model.train()`;
 2. сделайте предсказание (forward pass):
`y_pred = model(X_train)`;
 3. рассчитайте потери (loss):
`loss = criterion(y_pred, y_train)`;
 4. обнулите градиенты: `optimizer.zero_grad()`;
 5. выполните обратное распространение ошибки:
`loss.backward()`;
 6. сделайте шаг оптимизации: `optimizer.step()`.

5. Оценка модели (Evaluation)

- переведите модель в режим оценки: `model.eval()`;
- используйте `with torch.no_grad():`, чтобы отключить расчет градиентов;
- сделайте предсказания на тестовых данных;
- преобразуйте выходные данные (логиты) в предсказанные классы (например,

с помощью `torch.argmax` или проверки порога > 0);

- рассчитайте метрики (accuracy, f1-score и т.д.), используя `sklearn.metrics`.

Вариант 11 Анализ маркетинговой кампании банка

- Bank Marketing UCI
- **Задача:** предсказать, согласится ли клиент на вклад (бинарная классификация).
- **Архитектура:**
 - входной слой;
 - один скрытый слой с 20 нейронами (ReLU);
 - выходной слой с 1 нейроном (Sigmoid).
- **Эксперимент:** попробуйте использовать оптимизатор RMSprop вместо Adam.

Сравните итоговый F1-score.

Код программы:

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, f1_score, classification_report

# 1. Импорт библиотек и подготовка данных
# Указываем полный путь к файлу
file_path = "G:/ЛАБЫ 3 КУРС/ОМО/ml_as66/reports/Nastya/4/src/bank.csv"

try:
    # Загрузка датасета с указанием полного пути
    print(f"Загрузка данных из: {file_path}")
    data = pd.read_csv(file_path, delimiter=',')
    print("Данные успешно загружены!")
    print(f"Размер датасета: {data.shape}")
    print(f"Колонки: {data.columns.tolist()}")

except FileNotFoundError:
    print(f"Файл не найден по пути: {file_path}")
    print("Проверьте путь к файлу и попробуйте снова.")
    exit()

# Предобработка данных
# Кодирование категориальных переменных
categorical_columns = ['job', 'marital', 'education', 'default', 'housing', 'loan',
                       'contact', 'month', 'outcome', 'deposit']
label_encoders = {}

print("Кодирование категориальных переменных...")
for col in categorical_columns:
    if col in data.columns:
        le = LabelEncoder()
        data[col] = le.fit_transform(data[col].astype(str))
        label_encoders[col] = le
    print(f"Закодирована колонка: {col}")

# Разделение на признаки и целевую переменную
X = data.drop('deposit', axis=1)
y = data['deposit']

print(f"Признаки: {X.shape}, Целевая переменная: {y.shape}")

# Стандартизация числовых признаков
scaler = StandardScaler()
numerical_columns = ['age', 'balance', 'day', 'duration', 'campaign', 'pdays',
                     'previous']
```

```

X[numerical_columns] = scaler.fit_transform(X[numerical_columns])

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)

print(f"Обучающая выборка: {X_train.shape}")
print(f"Тестовая выборка: {X_test.shape}")

# Преобразование в тензоры PyTorch
X_train_tensor = torch.tensor(X_train.values, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test.values, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train.values, dtype=torch.float32).view(-1, 1)
y_test_tensor = torch.tensor(y_test.values, dtype=torch.float32).view(-1, 1)

print("Данные преобразованы в тензоры PyTorch")

# 2. Определение архитектуры нейронной сети
class BankMarketingNN(nn.Module):
    def __init__(self, input_size):
        super(BankMarketingNN, self).__init__()
        self.layer1 = nn.Linear(input_size, 20)
        self.relu = nn.ReLU()
        self.output = nn.Linear(20, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.layer1(x)
        x = self.relu(x)
        x = self.output(x)
        x = self.sigmoid(x)
        return x

# 3. Инициализация модели, функции потерь и оптимизаторов
input_size = X_train.shape[1]
print(f"Размер входного слоя: {input_size}")

model_adam = BankMarketingNN(input_size)
model_rmsprop = BankMarketingNN(input_size)

criterion = nn.BCELoss() # Binary Cross Entropy для бинарной классификации

# Два оптимизатора для сравнения
optimizer_adam = optim.Adam(model_adam.parameters(), lr=0.001)
optimizer_rmsprop = optim.RMSprop(model_rmsprop.parameters(), lr=0.001)

print("Модели и оптимизаторы инициализированы")

# 4. Цикл обучения
def train_model(model, optimizer, X_train, y_train, epochs=100):
    model.train()
    losses = []

    for epoch in range(epochs):
        # Forward pass
        y_pred = model(X_train)
        loss = criterion(y_pred, y_train)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        losses.append(loss.item())

        if (epoch + 1) % 20 == 0:
            print(f'Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}')

    return losses

print("\n" + "="*50)

```

```

print("ОБУЧЕНИЕ С ОПТИМИЗАТОРОМ ADAM")
print("=="*50)
losses_adam = train_model(model_adam, optimizer_adam, X_train_tensor, y_train_tensor,
epochs=100)

print("\n" + "=="*50)
print("ОБУЧЕНИЕ С ОПТИМИЗАТОРОМ RMSPROP")
print("=="*50)
losses_rmsprop = train_model(model_rmsprop, optimizer_rmsprop, X_train_tensor,
y_train_tensor, epochs=100)

# 5. Оценка модели
def evaluate_model(model, X_test, y_test):
    model.eval()
    with torch.no_grad():
        y_pred_proba = model(X_test)
        y_pred = (y_pred_proba > 0.5).float()

        accuracy = accuracy_score(y_test.numpy(), y_pred.numpy())
        f1 = f1_score(y_test.numpy(), y_pred.numpy())

    return accuracy, f1, y_pred

print("\n" + "=="*50)
print("ОЦЕНКА МОДЕЛЕЙ")
print("=="*50)

print("\nМодель с Adam:")
accuracy_adam, f1_adam, y_pred_adam = evaluate_model(model_adam, X_test_tensor,
y_test_tensor)
print(f"Accuracy: {accuracy_adam:.4f}")
print(f"F1-Score: {f1_adam:.4f}")

print("\nМодель с RMSprop:")
accuracy_rmsprop, f1_rmsprop, y_pred_rmsprop = evaluate_model(model_rmsprop,
X_test_tensor, y_test_tensor)
print(f"Accuracy: {accuracy_rmsprop:.4f}")
print(f"F1-Score: {f1_rmsprop:.4f}")

# Сравнение результатов
print("\n" + "=="*50)
print("СРАВНЕНИЕ РЕЗУЛЬТАТОВ")
print("=="*50)
print(f"Adam Optimizer:")
print(f"  Accuracy: {accuracy_adam:.4f}")
print(f"  F1-Score: {f1_adam:.4f}")

print(f"\nRMSprop Optimizer:")
print(f"  Accuracy: {accuracy_rmsprop:.4f}")
print(f"  F1-Score: {f1_rmsprop:.4f}")

print(f"\nРазница в F1-Score: {abs(f1_adam - f1_rmsprop):.4f}")

if f1_adam > f1_rmsprop:
    print("  Adam показал лучший результат по F1-Score")
elif f1_rmsprop > f1_adam:
    print("  RMSprop показал лучший результат по F1-Score")
else:
    print("  Оба оптимизатора показали одинаковый результат")

# Детальная классификационная отчетность
print("\n" + "=="*50)
print("ДЕТАЛЬНЫЕ ОТЧЕТЫ КЛАССИФИКАЦИИ")
print("=="*50)

print("\nОтчет для Adam:")
print(classification_report(y_test_tensor.numpy(), y_pred_adam.numpy(), target_names=['No
Deposit', 'Deposit']))

print("\nОтчет для RMSprop:")
print(classification_report(y_test_tensor.numpy(), y_pred_rmsprop.numpy(),

```

```
target_names=['No Deposit', 'Deposit']))

print("\n" + "="*50)
print("ОБУЧЕНИЕ ЗАВЕРШЕНО!")
print("="*50)
```

Результат программы:

```
Загрузка данных из: G:/ЛАБЫ 3 КУРС/ОМО/ml_as66/reports/Nastya/4/src/bank.csv
Данные успешно загружены!
Размер датасета: (11162, 17)
Колонки: ['age', 'job', 'marital', 'education', 'default', 'balance', 'housing', 'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays', 'previous', 'poutcome', 'deposit']
Кодирование категориальных переменных...
Закодирована колонка: job
Закодирована колонка: marital
Закодирована колонка: education
Закодирована колонка: default
Закодирована колонка: housing
Закодирована колонка: loan
Закодирована колонка: contact
Закодирована колонка: month
Закодирована колонка: poutcome
Закодирована колонка: deposit
Признаки: (11162, 16), Целевая переменная: (11162,)
Обучающая выборка: (8929, 16)
Тестовая выборка: (2233, 16)
Данные преобразованы в тензоры PyTorch
Размер входного слоя: 16
Модели и оптимизаторы инициализированы

=====
ОБУЧЕНИЕ С ОПТИМИЗАТОРОМ ADAM
=====
Epoch [20/100], Loss: 0.6594
Epoch [40/100], Loss: 0.6253
Epoch [60/100], Loss: 0.5910
Epoch [80/100], Loss: 0.5536
Epoch [100/100], Loss: 0.5260

=====
ОБУЧЕНИЕ С ОПТИМИЗАТОРОМ RMSPROP
=====
Epoch [20/100], Loss: 0.6058
Epoch [40/100], Loss: 0.5496
Epoch [60/100], Loss: 0.5203
Epoch [80/100], Loss: 0.5020
Epoch [100/100], Loss: 0.4891

=====
ОЦЕНКА МОДЕЛЕЙ
=====

Модель с Adam:
Accuracy: 0.7559
F1-Score: 0.7389

Модель с RMSprop:
Accuracy: 0.7895
F1-Score: 0.7764
```

```

=====
СРАВНЕНИЕ РЕЗУЛЬТАТОВ
=====
Adam Optimizer:
  Accuracy: 0.7559
  F1-Score: 0.7389

RMSprop Optimizer:
  Accuracy: 0.7895
  F1-Score: 0.7764

Разница в F1-Score: 0.0375
RMSprop показал лучший результат по F1-Score

=====
ДЕТАЛЬНЫЕ ОТЧЕТЫ КЛАССИФИКАЦИИ
=====

Отчет для Adam:
      precision    recall  f1-score   support

   No Deposit      0.76      0.78      0.77      1175
    Deposit      0.75      0.73      0.74      1058

   accuracy
  macro avg      0.76      0.75      0.75      2233
weighted avg      0.76      0.76      0.76      2233


Отчет для RMSprop:
      precision    recall  f1-score   support

   No Deposit      0.80      0.81      0.80      1175
    Deposit      0.78      0.77      0.78      1058

   accuracy
  macro avg      0.79      0.79      0.79      2233
weighted avg      0.79      0.79      0.79      2233

```

Вывод: построила, обучила и оценила многослойный перцептрон (MLP) для решения задачи классификации.

